

ĐẠI HỌC QUỐC GIA HÀ NỘI
ĐẠI HỌC CÔNG NGHỆ



KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO: CÁC VẤN ĐỀ HIỆN ĐẠI CỦA CÔNG NGHỆ THÔNG TIN

An Empirical Study of Automated Unit Test Generation for Python

Stephan Lukasczyk · Florian Kroiß · Gordon Fraser

Sinh viên:

Trần Tất Việt (L) – 21020132

Trịnh Đức Thành – 21020792

Đinh Thế Vương – 21020424

Vũ Minh Tuấn – 21020093

Giảng viên:

Nguyễn Đức Anh

Mã lớp học phần:

INT3507 7

Hà Nội, ngày 19 tháng 12 năm 2024

PHÂN CÔNG CÔNG VIỆC

Tên	MSSV	Tasks	Tỉ lệ
Trần Tất Việt (Nhóm trưởng)	21020132	- Lên kế hoạch phân chia phần làm slide và báo cáo. - Giám sát và đảm bảo tiến độ của các thành viên. - Viết phần cơ sở lý thuyết, thuật toán WS, Assertion, RQ3 cho slide và báo cáo. - Lên layout cho báo cáo, chỉnh sửa styles cho báo cáo.	26
Trịnh Đức Thành	21020792	Viết báo cáo và slide phần toán tử đột biến, thuật toán Many Independent Objective, RQ2 cho slide và báo cáo.	22
Đinh Thế Vương	21020424	- Thuyết trình - Viết báo cáo và slide phần Mở đầu, Thuật toán tiến hóa, thuật toán MOSA, thuật toán Feedback-directed random generation, RQ1 cho slide và báo cáo.	24
Vũ Minh Tuấn	21020093	- Đưa ví dụ trực quan cho thuật toán tiến hóa - Viết báo cáo và slide phần thuật toán DynaMOSA, phần branch coverage & fitness score - Làm demo và sample cho công cụ	28

LỜI CAM ĐOAN

Chúng em xin cam kết bài báo cáo này được viết bởi các thành viên trong nhóm và dưới sự hướng dẫn của thầy Nguyễn Đức Anh.

Bài báo cáo này là thành quả nghiên cứu của các thành viên trong nhóm, không sao chép và không sử dụng các mô hình AI để dịch hay tạo sinh bài báo cáo.

Nếu có gì sai, chúng em xin hoàn toàn chịu trách nhiệm.

Hà Nội, ngày 19 tháng 12 năm 2024

Các thành viên nhóm

DANH MỤC HÌNH ẢNH

Hình 1. Ví dụ 2 đoạn mã ngôn ngữ tĩnh (trái) và động (phải)	12
Hình 2: Mô hình thực thi trong Pynguin.....	13
Hình 3: Sơ đồ hoạt động của thuật toán tiến hóa.....	17
Hình 4: Một class kiểm thử với hai test case tiền crossover.....	19
Hình 5: Hai test case sau khi thực hiện crossover	20
Hình 6: Ví dụ về đồ thị phụ thuộc dòng điều khiển.....	26
Hình 7: Một đoạn mã để sinh test với assertion	30
Hình 8: Một số test case cho đoạn code trên	30
Hình 9: Một số test case với exception cho đoạn code trên	31
Hình 10: Độ phủ mã theo thời gian với thông tin kiểu dữ liệu.....	34
Hình 11: Phân phối effect-size với mỗi project (DynaMOSA)	35
Hình 12: Phân bố mutation score đối với configurations trên.....	38
Hình 13: Mối tương quan giữa mutation score và độ phủ.....	39
Hình 14: Đoạn mã kiểm tra kiểu dữ liệu hỗ trợ của Pynguin.	43

MỤC LỤC

PHÂN CÔNG CÔNG VIỆC	2
LỜI CAM ĐOAN	3
DANH MỤC HÌNH ẢNH.....	4
GLOSSARY.....	6
CHƯƠNG 1. MỞ ĐẦU	8
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	10
2.1. Giới thiệu về kiểm thử	10
2.1.1. Định nghĩa về kiểm thử	10
2.1.2. Vai trò của kiểm thử	10
2.1.3. Mục đích của kiểm thử	10
2.2. Ngôn ngữ động	10
2.3. Unit testing.....	11
2.3.1. Định nghĩa Unit testing.....	11
2.3.2. Lợi ích của Unit testing	11
2.3.3. Các phương pháp sinh Unit test.....	11
2.4. Tầm quan trọng kiểu dữ liệu.....	12
2.5. Giới thiệu về Pynguin	13
CHƯƠNG 3. THUẬT TOÁN	15
3.1. Thuật toán FDRG.....	15
3.2. Thuật toán tiến hóa	16
3.2.1. Khái quát về thuật toán tiến hóa	16
3.2.2. Điều kiện dừng của thuật toán	18
3.2.3. Toán tử cho thuật toán tiến hóa	18
3.2.3.1. Toán tử Crossover.....	18
3.2.3.2. Toán tử Mutation	20
3.2.4. Phương thức đánh giá trên thuật toán tiến hoá	21
3.2.5. Các thuật toán tiến hóa tiêu biểu	22
3.2.5.1. Thuật toán Whole Suite	22

3.2.5.2. Thuật toán MOSA.....	24
3.2.5.3. Thuật toán DynaMOSA.....	26
3.2.5.4. Thuật toán MIO.....	27
CHƯƠNG 4. ASSERTION & MUTATION	29
4.1. Định nghĩa và ví dụ.....	29
4.2. Exceptions.....	31
4.3. Cơ chế kiểm thử với assertion	32
CHƯƠNG 5. THỰC NGHIỆM	33
5.1. RQ1: So sánh hiệu quả của các thuật toán được đề cập.	34
5.2. RQ2: Ảnh hưởng của thông tin kiểu dữ liệu	35
5.3. RQ3: Hiệu quả Assertion trong việc phát hiện lỗi.....	36
5.3.1. Một số kiến thức liên quan	36
5.3.1.1. Hệ số tương quan Pearson	36
5.3.1.2. Giá trị p-value	37
5.3.2. Kết quả thực nghiệm.....	38
CHƯƠNG 6. KẾT LUẬN	40
6.1. Hạn chế Pynguin.....	40
6.2. Nhận xét.....	41
CHƯƠNG 7. PHỤ LỤC	42
7.1. Ý nghĩa của kiểm thử.....	42
7.2. Cách sinh giá trị Assertion.....	42
7.3. Thống kê về các kiểu dữ liệu hỗ trợ trong Pynguin	42
TÀI LIỆU THAM KHẢO	44

GLOSSARY

Bảng thuật ngữ xác định các thuật ngữ cụ thể được dùng trong bài báo cáo.

Thuật ngữ	Định nghĩa
Kiểm thử	Là quá trình kiểm tra xem 1 hàm, module, hay chức năng có hoạt động đúng theo mong muốn hay không.
Kiểu dữ liệu	Loại dữ liệu mà 1 biến có thể lưu trữ như số nguyên, chuỗi, mảng,...
Cá thể	Là 1 lời giải khả thi cho một bài toán tối ưu. Có thể là các test case, module mã nguồn...
Quần thể	Tập hợp của nhiều cá thể trong 1 thể hệ.
Fitness Score	Chỉ số đánh giá mức độ hiệu quả của 1 cá thể cho bài toán.
Độ phủ	Mức độ được bao quát của mã nguồn với 1 ca kiểm thử.
Test case (Ca kiểm thử)	Một bộ test hoặc một số bước để kiểm tra chức năng của hàm, module.
Test suite	Một tập hợp các ca kiểm thử (test case) được nhóm lại với nhau nhằm thực thi và đánh giá một chức năng cụ thể của chương trình.

CHƯƠNG 1. MỞ ĐẦU

Hiện nay, lĩnh vực công nghệ nói chung và ngành phát triển phần mềm nói riêng đang phát triển rất nhanh. Càng nhiều phần mềm mới được phát triển thì công việc kiểm thử cũng càng tăng. Quy trình kiểm thử phần mềm được chia ra nhiều bước:

- Kiểm thử đơn vị: kiểm thử các hàm, các module đơn giản.
- Kiểm thử tích hợp: kiểm tra sự tương tác giữa các module hoặc thành phần.
- Kiểm thử hệ thống: kiểm tra toàn bộ hệ thống phần mềm.
- Kiểm thử chấp nhận: xác định xem hệ thống cuối cùng có đúng với yêu cầu của khách hàng hay không.

Trong đó, kiểm thử đơn vị là bước đầu tiên nhưng cũng vô cùng quan trọng để đảm bảo rằng các đoạn mã khi tích hợp với nhau vẫn chạy đúng như mong muốn. Tuy nhiên, trong các hệ thống lớn, nếu chỉ kiểm thử thủ công thì sẽ vô cùng khó khăn cho những người kiểm thử. Do đó việc kiểm thử tự động và bán tự động được chú trọng. Muốn kiểm thử được thì ta cần có đầu vào là các hàm, module, và đầu ra là các bộ test case. Ngày nay có rất nhiều công cụ kiểm thử sinh các test case tự động như JUnit, EvoSuite,.. Các công cụ này hoạt động tốt, sinh ra các bộ test hiệu quả nhưng mới chỉ phù hợp cho các ngôn ngữ lập trình tĩnh như Java khi mà các kiểu dữ liệu được xác định ngay từ khi viết mã. Còn với các ngôn ngữ lập trình động như Python hay JavaScript thì chưa đáp ứng được.

Đây cũng chính là vấn đề được đặt ra trong bài nghiên cứu: việc tạo các ca kiểm thử tự động cho ngôn ngữ lập trình động Python. Hiện nay, nhiều ngôn ngữ động được phát triển và được ưu chuộng hơn so với các ngôn ngữ tĩnh bởi nó cho phép lập trình viên phát triển dự án một cách nhanh chóng. Nhưng mặt khác chúng cũng mang nhiều khó khăn trong việc kiểm thử, làm giảm chất lượng mã nguồn khi không xác định rõ kiểu dữ liệu của đối tượng từ ban đầu. Bài báo này đã đưa ra một số phương pháp nhằm vượt qua các vấn đề của ngôn ngữ động, đó là sử dụng các thuật toán tiến hóa và thuật toán ngẫu nhiên để tự động sinh các ca kiểm thử cho hàm, module.

Các nội dung chính mà bài nghiên cứu đưa ra:

- Các thuật toán có tiềm năng sinh ra được các ca kiểm thử với độ hiệu quả cao trong việc bao phủ mã nguồn và phát hiện lỗi.

- Các thử nghiệm đưa ra kết quả khả quan cho các thuật toán trong việc tạo ca kiểm thử.

Nội dung chính của báo cáo được trình bày với các chương như sau:

- Chương 1: Mở đầu, trình bày vấn đề và nội dung chính của bài báo.
- Chương 2: Các cơ sở lý thuyết, kiến thức nền tảng trong bài báo.
- Chương 3: Các thuật toán được sử dụng.
- Chương 4: Assertion & Mutation
- Chương 5: Thử nghiệm
- Chương 6: Kết luận
- Chương 7: Phụ lục

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Chương này sẽ tập trung khái quát hoá các khái niệm cơ sở lý thuyết liên quan đến kiểm thử, định nghĩa và một số ví dụ về ngôn ngữ động, Unit testing, tầm quan trọng của kiểu dữ liệu, việc xác định kiểu dữ liệu trong kiểm thử và giới thiệu về framework Pynguin.

2.1. Giới thiệu về kiểm thử

2.1.1. Định nghĩa về kiểm thử

Kiểm thử là một loạt các hoạt động nhằm tìm kiếm và phát hiện ra các lỗi của phần mềm, đảm bảo phần mềm chính xác, đúng và đầy đủ theo yêu cầu của khách hàng, yêu cầu của sản phẩm đã đặt ra, như yêu cầu chức năng, yêu cầu phi chức năng, bảo mật, tính tương thích trên các hệ điều hành khác nhau.

2.1.2. Vai trò của kiểm thử

Kiểm thử là quy trình bắt buộc trong mọi dự án phát triển và duy trì phần mềm hiện nay, với vai trò giám sát chất lượng sản phẩm trong khâu phát triển, nhằm tránh những lỗi và rủi ro có thể phát sinh. Ngoài ra việc kiểm thử cũng giúp tối thiểu được chi phí sửa chữa sản phẩm sau khi đã được phát hành. Vì những lý do đó, kiểm thử đóng một vai trò quan trọng và không thể bỏ trong quá trình phát triển sản phẩm. Thực tế, chi phí dành cho kiểm thử chiếm phần lớn chi phí phát triển dự án.

2.1.3. Mục đích của kiểm thử

Kiểm thử được sinh ra với các mục đích như kiểm tra sản phẩm đã đạt chất lượng mong đợi chưa, hay đã thoả mãn nghiệp vụ khách hàng chưa. Ngoài ra, kiểm thử còn giúp đánh giá mức độ tương thích của sản phẩm trên các hệ thống và hệ điều hành khác nhau, đảm bảo mức độ trơn tru và tương thích trên các môi trường riêng biệt. Kiểm thử cũng đóng một phần quan trọng trong việc đảm bảo trải nghiệm người dùng, giúp sản phẩm đạt hiệu quả doanh thu cao nhất.

2.2. Ngôn ngữ động

Ngôn ngữ động là các ngôn ngữ lập trình mà ở đó, kiểu dữ liệu chỉ được gán khi chạy chương trình. Các ngôn ngữ này có mặt lợi là giúp người lập trình không phải xác định kiểu dữ liệu trong quá trình viết mã, đồng thời có thể gán thêm nhiều kiểu dữ liệu lên các

biến. Tuy nhiên, vì ngôn ngữ lập trình động xem tất cả các vai trò của dữ liệu trong chương trình là ngang nhau, do vậy các phép toán không đúng (như là cộng các tên, hay là xếp thứ tự các số theo thứ tự đánh vần) sẽ không tạo ra các lỗi cho đến lúc nó được thi hành. Một số ngôn ngữ động có thể kể đến hiện giờ như Python, Javascript, PHP, Ruby, etc.

2.3. Unit testing

2.3.1. Định nghĩa Unit testing

Unit testing, hay còn gọi là kiểm thử đơn vị, là loại kiểm thử phần mềm mà trong đó các Unit (đơn vị) nhỏ nhất của phần mềm được kiểm thử như các Class (lớp), Function (hàm), Method (phương thức). Trong đó, mỗi Unit test kiểm tra và trả về các giá trị kỳ vọng mong muốn, và các lỗi phát sinh không mong muốn, và thường được thực hiện bởi các lập trình viên.

2.3.2. Lợi ích của Unit testing

Unit testing là việc kiểm thử các đơn vị nhỏ nhất của một bộ mã nguồn, vì vậy việc viết Unit test sẽ giúp tham chiếu đến bất cứ khi nào có sự thay đổi mã nguồn của các đơn vị đó, có thể kiểm tra được lỗi từ việc thay đổi đó. Ngoài ra, Unit testing cho phép kiểm thử các thành phần riêng lẻ mà không cần đợi các thành phần khác hoàn thành. Điều này cũng giúp việc khoanh vùng và vá lỗi được tốt và hiệu quả hơn. Đồng thời, chính vì việc kiểm thử các đơn vị nhỏ nhất nên thông thường chi phí viết Unit test và sửa chữa lỗi tại giai đoạn này sẽ thấp hơn so với các giai đoạn kiểm thử tích hợp, kiểm thử hệ thống, etc. Điều này giúp tiết kiệm được chi phí bảo trì sản phẩm và tăng độ tin cậy cho mã nguồn.

2.3.3. Các phương pháp sinh Unit test

Một số cách tiếp cận chính cho việc sinh Unit test có thể bao gồm các chuỗi ngẫu nhiên hoặc sử dụng các thuật toán tìm kiếm di truyền. Trong đó, kiểm thử ngẫu nhiên sẽ khởi tạo các chuỗi gọi đến các constructors hoặc methods một cách ngẫu nhiên, tức một chuỗi sẽ có thể khởi tạo đến một Object (đối tượng) mới, hoặc gọi đến Method (phương thức) trong Object đó một cách ngẫu nhiên và đưa vào chuỗi. Mục đích của phương pháp này nhằm phát hiện các ngoại lệ tiềm ẩn, và không được khai báo trong documentation (sẽ được làm rõ hơn ở chương Assertion), và tái sử dụng như các regression tests (sẽ được làm rõ hơn ở chương Assertion).

Trong khi đó, đối với cách tiếp cận tìm kiếm di truyền, các thuật toán tiến hoá sẽ được sử dụng nhằm sinh các đoạn test với mục tiêu tối ưu hoá độ phủ code (sẽ được làm rõ hơn ở chương Evolutionary Algorithms). Mặc dù hai phương pháp này đều giúp khảo sát các đoạn mã nguồn, tuy nhiên cách tiếp cận này vượt trội hơn so với phương pháp ngẫu nhiên, với việc được tích hợp thêm các hàm tối ưu hoá tìm kiếm thay vì việc gọi ngẫu nhiên hàm.

2.4. Tầm quan trọng kiểu dữ liệu

Kiểu dữ liệu giúp cho việc viết mã trở nên chặt chẽ, quy củ, giúp các biến, đối tượng được khai báo và sử dụng đúng với kiểu dữ liệu của chúng. Khi kiểm thử, các công cụ sẽ sinh ra các test case ứng với kiểu dữ liệu đó mà không cần phải sinh các test case với kiểu dữ liệu khác để thử.

Ví dụ ta có 2 đoạn mã Java (bên trái) và Python (bên phải) sau đây:

<pre>class Foo { Foo(Bar b) { ... } void doFoo(Bar b) { ... } } class Bar { Bar() { ... } Bar doBar(Bar b) { ... } }</pre>	<pre>class Foo: def __init__(self, b): ... def do_foo(self, b): ... class Bar: def __init__(self): ... def do_bar(self, b): ...</pre>
--	---

Hình 1. Ví dụ 2 đoạn mã ngôn ngữ tĩnh (trái) và động (phải)

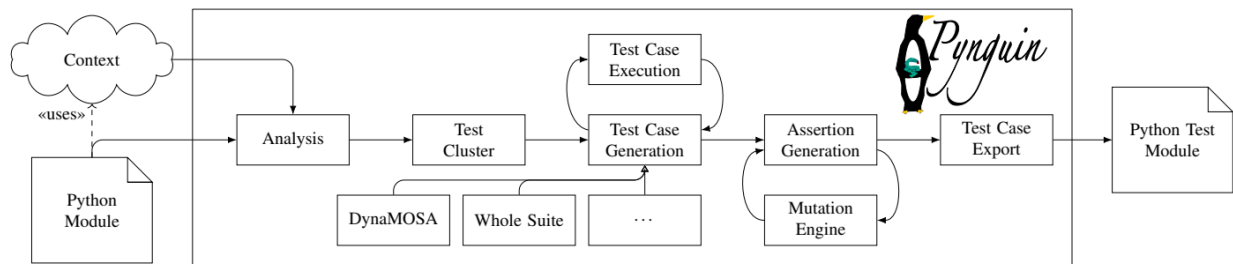
Với Java, giả sử Foo là lớp đang được kiểm tra, lớp này phụ thuộc vào đối tượng kiểu Bar. Do đó, lời gọi đầu tiên chỉ có thể là khởi tạo Bar vì các phương thức khác đều cần tham số. Vì đã có hàm tạo Bar nên tiếp theo có thể chọn hàm tạo Foo hoặc phương thức doBar.

Còn với mã Python, do không có kiểu của tham số nên tất cả cá hàm và phương thức đều có thể được chọn. Nếu có nhiều các đối tượng hơn, số lượng hàm, phương thức có thể được gọi sẽ tăng rất nhiều. Điều này trái với mong muốn là hàm tạo của Foo và doFoo chỉ cần đối tượng Bar. Do ko có thông tin kiểu, nên tất cả các trường hợp kiểm tra như vậy đều có thể được tạo và chỉ thất bại nếu gặp ngoại lệ.

2.5. Giới thiệu về Pynguin

Pynguin là một framework được xây dựng trên mã nguồn Python, dùng để tạo các ca kiểm thử cho mã nguồn Python. Pynguin được phát triển với mục tiêu tạo các ca kiểm thử tự động cho mã nguồn động (thiếu kiểu dữ liệu) như Python.

Quá trình thực thi trong Pynguin được mô tả dưới hình sau:



Hình 2: Mô hình thực thi trong Pynguin

- **Context và Python Module:** Pynguin lấy đầu vào là 1 **Python Module** và các **Context** (ngữ cảnh) bao gồm điều kiện dừng chương trình, mục tiêu kiểm thử,..
- **Analysis:** Sau khi nhận đầu vào, Pynguin sẽ phân tích mã nguồn để làm rõ các yếu tố cần kiểm thử và các đoạn mã dễ gây lỗi.
- **Test Cluster:** Đây là nơi sẽ nhóm các trường hợp kiểm thử có tính tương đồng cao và tối ưu hóa chúng nhằm đảm bảo tính đầy đủ và hiệu quả cho quá trình kiểm thử.
- **Test Case Generation:** Pynguin sẽ sinh ra các test cases dựa trên các thuật toán tiến hóa được sử dụng như DynaMOSA, Whole Suite,..
- **Test case Execution:** Đây là nơi các test cases được sinh ra sẽ được thực thi.
- **Assertion Generation:** Sinh ra các assertions nhằm so sánh giá trị thực thi của module với giá trị thực tế kỳ vọng.
- **Mutation Engine:** Làm giảm số lượng assertions, chỉ giữ lại các assertions có giá trị.

- **Test Case Export:** Lọc ra các test case đạt điều kiện đầu ra.
- **Python Test Module:** Với mỗi test case, Pynguin sẽ tạo ra một module test của test case đó để người dùng có thể chạy thực thi.

CHƯƠNG 3. THUẬT TOÁN

Phần này sẽ đề cập đến các thuật toán tiêu biểu được nghiên cứu trong bài báo, bao gồm thuật toán FDRG (Feedback-directed random generation), 4 thuật toán tiến hóa: Whole-Suite, MOSA (Many Objective Sorting Algorithm), DynaMOSA (Dynamic Target Selection MOSA), MIO (Many Independent Objective).

3.1. Thuật toán FDRG

Thuật toán FDRG (Feedback-directed random generation) hay thuật toán tạo ngẫu nhiên theo hướng phản hồi sẽ chạy thực thi các bộ test đầu vào ngẫu nhiên để lấy phản hồi (khu vực lỗi, độ phủ,...) sau đó cải thiện các bộ test sinh ra tiếp theo dựa trên phản hồi để đạt được kết quả tối ưu.

Dưới đây là mã giả cho thuật toán FDRG:

Input: Điều kiện dừng C , 1 tập module c .

Output: Một cặp chuỗi không lỗi s_n và chuỗi lỗi s_e .

```
1   $s_e \leftarrow \{\}$ ;
2   $s_n \leftarrow \{\}$ ;
3  while  $\neg C$  do
4       $m(T_1 \dots T_k) \leftarrow \text{RandomPublicMethod}(c)$ ;
5       $\langle S, V \rangle \leftarrow \text{RandomSeqsAndVals}(s_n, T_1 \dots T_k)$ ;
6       $N \leftarrow \text{Extend}(m, S, V)$ ;
7      if  $N \in s_n \cup s_e$  then
8          Continue
9       $\langle \sim o, v \rangle \leftarrow \text{Execute}(N)$ ;
10     if  $v$  then
11          $s_e \leftarrow s_e \cup \{N\}$ ;
12     else
13          $s_n \leftarrow s_n \cup \{N\}$ ;
14 return  $\langle s_n, s_e \rangle$ 
```

Luồng hoạt động:

- **Bước 1:** Khởi tạo 2 bộ test-suite rỗng cho test pass (s_n) và test fail (s_e)

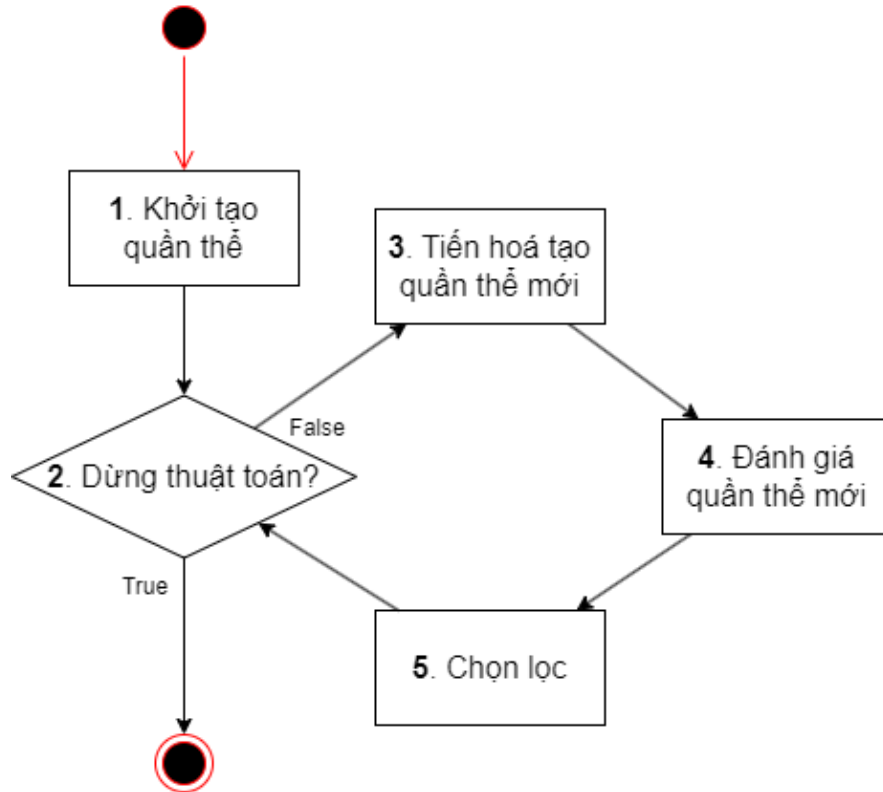
- **Bước 2:** Trong mỗi vòng lặp (thế hệ), kiểm tra điều kiện dừng C rồi:
- **Bước 3:** Chọn 1 phương thức ngẫu nhiên N chưa xuất hiện trong 2 bộ test suite trên trong tập c rồi thực thi với tham số ngẫu nhiên
- **Bước 4:**
 - Nếu thực thi thành công, thêm N vào bộ test pass sn
 - Nếu có lỗi trong quá trình thực thi, thêm N vào bộ test fail se

3.2. Thuật toán tiến hóa

3.2.1. Khái quát về thuật toán tiến hóa

Thuật toán tiến hóa lấy cảm hứng từ thuyết tiến hóa của các cá thể sinh vật trong quần thể trong tự nhiên.

- **Input:** Thuật toán lấy đầu vào là các tham số chính, phạm vi của các biến. Với từng thuật toán tiến hóa khác nhau, đầu vào sẽ có sự khác biệt nhất định.
- **Output:** Giá trị tối ưu nhất đạt được của các biến, hàm mục tiêu
- **Bước 1:** Khởi tạo quần thể ban đầu $P(t)$ với $t = 0$ với các cá thể ngẫu nhiên.
- **Bước 2:** Kiểm tra xem các cá thể trong quần thể đã tối ưu, đạt điều kiện đầu ra hay chưa. Điều kiện đầu ra có thể là hết tài nguyên, đạt đến giới hạn mà đầu vào đặt ra, độ phủ đạt được. Nếu đạt điều kiện, trả về kết quả đầu ra. Nếu chưa đạt, chuyển sang bước 3.
- **Bước 3:** Tạo ra các cá thể cho quần thể mới dựa vào các toán tử Lai tạo (Crossover), Đột biến (Mutation). Quần thể thế hệ kế tiếp $P(t)$, $t = t + 1$ sẽ được tiếp tục vòng lặp đánh giá, chọn lọc cho đến khi đạt điều kiện đầu ra.
- **Bước 4:** Đánh giá các quần thể trong cá thể bằng các hàm đánh giá. Các hàm đánh giá sẽ có khác biệt giữa các thuật toán tiến hóa cụ thể khác nhau.
- **Bước 5:** Chọn lọc các cá thể tốt bằng các luật heuristic hoặc theo một xác suất nhất định.



Hình 3: Sơ đồ hoạt động của thuật toán tiến hóa

* **Lưu ý:** Các bước từ 2 - 5 có thể thay đổi vị trí tùy thuật toán cụ thể

Ưu điểm của thuật toán tiến hóa là giúp cải thiện dần dần các bộ test case để tăng độ phủ code, nâng cao độ hiệu quả của bộ test trong quá trình kiểm thử. Tuy nhiên, thuật toán tiến hóa cũng gặp nhiều nhược điểm:

- Tốn nhiều tài nguyên thời gian, bộ nhớ khi số lượng cá thể và thế hệ quần thể tăng cao.
- Hiệu suất thuật toán có thể không đạt được giá trị cao do nó phụ thuộc vào các tham số đầu vào. Nếu không tối ưu được tham số đầu vào, đầu ra của thuật toán sẽ không đạt được kết quả như mong muốn.
- Việc tạo cá thể mới bằng các phép lai tạo, đột biến còn mang tính ngẫu nhiên cao, dẫn đến việc các test case được tạo ra không đảm bảo được hiệu quả luôn cao.

3.2.2. Điều kiện dừng của thuật toán

Pynguin hỗ trợ các điều kiện dừng sau:

- `maximum_coverage`: Độ phủ tối đa cần đạt được
- `maximum_coverage_plateau`: Số lượng thể hệ tối đa mà độ phủ không thay đổi
- `maximum_iterations`: Số thể hệ (vòng lặp) tối đa
- `maximum_search_time`: Thời gian chạy thuật toán tối đa
- `test_execution_time_per_statement`: Thời gian chạy tối đa cho mỗi thuật toán
- `maximum_test_executions`: Số lượng test case tối đa được thực thi

Pynguin cho phép người dùng thay đổi các điều kiện này bằng tham số dòng lệnh. Các điều kiện trên có tên giống nhau cho tất cả các thuật toán của Pynguin và được ký hiệu là C trong phần 3.2.5.

3.2.3. Toán tử cho thuật toán tiến hóa

Các toán tử là thành phần cốt lõi trong các thuật toán tiến hóa, các quần thể muốn tạo ra những thể hệ tiếp theo đều cần phải lai tạo quần thể ban đầu để tạo ra các cá thể mới. Ngoài trừ Feedback-Directed Random Test Generation, tất cả các thuật toán được triển khai trong Pynguin đều là thuật toán tiến hóa và đều sử dụng 2 loại toán tử: Crossover và Mutation.

3.2.3.1. Toán tử Crossover

Crossover (lai tạo) hợp nhất các đặc tính di truyền của 2 cá thể để tạo ra một cá thể mới. Các thuật toán tiến hóa khác nhau có các cách định nghĩa cá thể khác nhau. DynaMOSA, MIO và MOSA coi mỗi test case là một cá thể trong khi Whole Suite coi một test suite (tập các test case) là một cá thể. Do đó, có sự khác biệt nhất định giữa 2 cách lai ghép test-suite crossover và test-case crossover.

Test-suite Crossover: Quá trình lai ghép sẽ theo các bước sau:

- Chọn ra 2 cá thể gốc.
- Chia cắt 2 cá thể tại cùng một vị trí tương đối.
- Hoán đổi phần sau của 2 cá thể.
- Ghép nối phần đầu và phần sau đã hoán đổi để tạo ra cá thể để tạo ra 2 cá thể con.

Do mỗi cá thể đang được xét là một bộ các test-case, sẽ không có sự phụ thuộc nào giữa chúng nên cá thể mới sinh ra sẽ luôn hợp lệ.

Quy trình lai ghép của test-case crossover cũng tương tự như test-suite crossover, tuy nhiên do các cá thể được xét là những test-case nên việc cắt và ghép nối các test-case sẽ có thể gặp một số vấn đề liên quan đến sự phụ thuộc giữa các hàm vào các biến khai báo.

Ở hình 8, phần sau của test-case 0 có biến str_1 được khai báo ở phần trước, tương tự với biến int_0 ở test_case 1. Nếu crossover tương tự test-suite crossover, test-case sẽ bị lỗi vì thiếu khai báo.

```
# Class under test
class Foo:
    def __init__(self, foo: str):
        self._foo = foo

    def bar(self, suffix: str) -> None:
        print(self._foo + suffix)

    def baz(self, repeat: int) -> None:
        print(self._foo * repeat)

# Test cases
def test_case_0():
    str_0 = "string a"
    foo_0 = Foo(str_0)
    str_1 = "string b"
    # <- Randomly chosen crossover point
    foo_0.bar(str_1)

def test_case_1():
    str_0 = "string c"
    foo_0 = Foo(str_0)
    int_0 = 1337
    # <- Randomly chosen crossover point
    foo_0.baz(int_0)
```

Hình 4: Một class kiểm thử với hai test case tiền crossover

Pynguin đề ra 2 cách giải quyết vấn đề này (hình 9):

- Với test-case crossover_0, biến int_0 và foo_1 đã được tạo mới.
- Với test-case crossover_1, biến str_1 được thay thế bằng str_0 cùng kiểu dữ liệu.

```

def test_case_after_crossover_0():
    # Statements from test_case_0
    str_0 = "string a"
    foo_0 = Foo(str_0) # Temporarily unused after crossover
    str_1 = "string b" # Temporarily unused after crossover
    # Statement from test_case_1 + repairing statements
    foo_1 = Foo(str_0)
    int_0 = 42
    foo_1.baz(int_0)

def test_case_after_crossover_1():
    # Statements from test_case_1
    str_0 = "string c"
    foo_0 = Foo(str_0)
    int_0 = 1337 # Temporarily unused after crossover
    # Statement from test_case_0
    foo_0.bar(str_0) # Used str_0 to satisfy parameter

```

Hình 5: Hai test case sau khi thực hiện crossover

3.2.3.2. Toán tử Mutation

Mutation (đột biến) có sự khác biệt trên test case và test suite

* **Đối với test case:** Có 3 toán tử mutation là remove, change và insert. Xác suất để mỗi toán tử này xảy ra trong một lần đột biến test case là $\frac{1}{3}$.

Toán tử chèn (insert):

- Với xác suất σ được truyền vào từ tham số, một câu lệnh mới được chèn vào vị trí ngẫu nhiên p trong test case. Các câu lệnh tiếp theo sẽ được chèn vào test case với cấp số mũ $\sigma^2, \sigma^3 \dots$. Quá trình này sẽ dừng lại nếu độ dài test case đạt đến giới hạn L (L là giới hạn từ tham số đầu vào).
- Câu lệnh được chèn vào có xác suất 50% là một phương thức từ module đang kiểm thử và 50% là một phương thức hoặc đối tượng từ test case đang đột biến

Toán tử thay đổi (change):

- Với một test case có độ dài l , mỗi câu lệnh sẽ có xác suất bị biến đổi là $1/l$.

- Đối với các giá trị số nguyên (int): Chọn một giá trị ngẫu nhiên từ phân phối chuẩn và thêm vào giá trị hiện tại tại $\alpha\Delta int$ trong đó Δint là một hằng số.
- Đối với các giá trị số thực (float): Thêm vào giá trị $\alpha\Delta float$ hoặc thay đổi số chữ số thập phân của giá trị hiện tại trong khoảng $[0, \alpha\Delta digits]$ trong đó $\Delta float$ và $\Delta digits$ là các hằng số.
- Đối với chuỗi ký tự (str), Byte (bytes) quá trình đột biến diễn ra tương tự như với các test case, các ký tự và các phần tử trong mảng byte tương tự như một test case.
- Đối với kiểu boolean: Đảo ngược true thành false và ngược lại.
- Tuple, List, Set, Dictionary có quá trình đột biến tương tự như chuỗi ký tự (str).
- Đặc biệt, khi đột biến một dictionary, có xác suất 1/2 để thay thế key hoặc value của dictionary.

Toán tử xóa (remove):

- Với một test case có độ dài l , mỗi câu lệnh sẽ có xác suất bị biến đổi là $1/l$.
- Sửa test case sau khi xóa: Khi một câu lệnh Si bị xóa, mọi tham chiếu đến nó cần phải được sửa lại cho hợp lệ. Đầu tiên Pynguin sẽ tìm kiếm một câu lệnh có cùng kiểu dữ liệu với Si để thay thế. Nếu không tồn tại câu lệnh nào như thế, tất cả những tham chiếu đến Si sẽ bị xóa.

*** Đối với test suite:**

Đối với một test suite T , mỗi test case sẽ được đột biến với tỉ lệ $1/|T|$ như trên. Sau khi quá trình đột biến hoàn tất, các test case mới sẽ được thêm ngẫu nhiên vào T với xác suất σ . Các test case tiếp theo sẽ được chèn vào test suite với cấp số mũ $\sigma^2, \sigma^3 \dots$. Quá trình này sẽ dừng lại nếu độ dài test suite đạt đến giới hạn N (N là giới hạn từ tham số đầu vào).

3.2.4. Phương thức đánh giá trên thuật toán tiến hoá

Mục tiêu của Pynguin là tạo ra bộ test case có độ phủ nhánh (branch coverage) lớn nhất trên toàn bộ module cần test. Để đánh giá xem một test case có phù hợp với mục tiêu này không, bài báo sử dụng 1 giá trị heuristic gọi là *branch distance*.

Branch distance thể hiện độ xa gần về giá trị thực tế hiện tại (của một biến trong một biểu thức điều kiện với giá trị thỏa mãn của biểu thức đó. Pynguin đã định nghĩa branch distance trên 10 toán tử trong biểu thức điều kiện như sau, trong đó $\text{lev}(a, b)$ là khoảng cách Levenstein của hai xâu a và b (số ký tự khác nhau giữa a và b):

$$\delta_{\equiv}(a, b) = \begin{cases} 0 & a \equiv b \\ |a - b| & a \not\equiv b \wedge \text{is_numeric}(a) \wedge \text{is_numeric}(b) \\ \text{lev}(a, b) & a \not\equiv b \wedge \text{is_string}(a) \wedge \text{is_string}(b) \\ \text{lev}(\text{decode}(a), \text{decode}(b)) & a \not\equiv b \wedge \text{is_bytes}(a) \wedge \text{is_bytes}(b) \\ \infty & \text{otherwise} \end{cases}$$

$$\delta_{<}(a, b) = \begin{cases} 0 & a < b \\ a - b + k & a \geq b \wedge \text{is_numeric}(a) \wedge \text{is_numeric}(b) \\ \infty & \text{otherwise} \end{cases}$$

$$\delta_{\leq}(a, b) = \begin{cases} 0 & a \leq b \\ a - b + k & a > b \wedge \text{is_numeric}(a) \wedge \text{is_numeric}(b) \\ \infty & \text{otherwise} \end{cases}$$

$$\delta_{>}(a, b) = \delta_{<}(b, a)$$

$$\delta_{\geq}(a, b) = \delta_{\leq}(b, a)$$

$$\delta_{\in}(a, b) = \begin{cases} 0 & a \in b \\ \min(\{\delta_{\equiv}(a, x) \mid x \in b\} \cup \{\infty\}) & \text{otherwise} \end{cases}$$

$$\delta_{\theta}(a, b) = \begin{cases} 0 & a \theta b \\ k & \text{otherwise} \end{cases} \quad \theta \in \{\neq, \notin, =, \in\}$$

Fitness score của từng cá thể trong mỗi thuật toán tiến hoá được sử dụng trong bài báo đều sử dụng giá trị branch distance $d(b, t)$ - branch distance trên nhánh b trong khi thực thi test case t . Chi tiết về hàm fitness của từng thuật toán sẽ được trình bày trong phần 3.2.5

3.2.5. Các thuật toán tiến hóa tiêu biểu

3.2.5.1. Thuật toán Whole Suite

Thuật toán Whole - Suite ứng dụng thuật toán di truyền, với đầu vào là các test suite (tập hợp 1 hoặc nhiều test cases) như 1 cá thể. Thuật toán cũng sử dụng thuật toán di truyền đơn điệu, tức các kết quả trả về sau mỗi lần lặp sẽ thỏa mãn ràng buộc thuật toán theo chiều

không đổi, nhằm đảm bảo tính đơn điệu toàn vẹn của kết quả. Ngoài ra, nhằm đánh giá chất lượng cá thể, thuật toán sử dụng tổng các khoảng cách nhánh là hàm fitness.

Dưới đây là tóm tắt về mã giả của thuật toán, cũng như giải thích sơ bộ về luồng hoạt động của thuật toán Whole-Suite tương ứng với các bước của thuật toán tiến hoá:

Input: Điều kiện dừng C , hàm fitness δ , kích thước quần thể p_s , hàm selection s_f , hàm crossover c_f , xác suất crossover c_b , hàm mutation m_f , xác suất mutation m_p

Output: Quần thể các cá thể tối ưu hoá P

```
1.  $P \leftarrow \text{GenerateRandomPopulation}(p_s)$ ;  
2.  $\text{PerformFitnessEvaluation}(\delta, P)$ ;  
3. while  $\neg C$  do  
4.    $NP \leftarrow \{\} \cup \text{Elitism}(P)$ ;  
5.   while  $|NP| < p_s$  do  
6.      $p_1, p_2 \leftarrow \text{Selection}(s_f, P)$ ;  
7.      $o_1, o_2 \leftarrow \text{Crossover}(c_f, c_b, p_1, p_2)$ ;  
8.      $\text{Mutation}(m_f, m_p, o_1)$ ;  
9.      $\text{Mutation}(m_f, m_p, o_2)$ ;  
10.     $\text{PerformFitnessEvaluation}(\delta, o_1)$ ;  
11.     $\text{PerformFitnessEvaluation}(\delta, o_2)$ ;  
12.    if  $\text{Best}(o_1, o_2)$  is better than  $\text{Best}(p_1, p_2)$  then  
13.       $NP \leftarrow NP \cup \{o_1, o_2\}$ ;  
14.    else  
15.       $NP \leftarrow NP \cup \{p_1, p_2\}$ ;  
16.   $P \leftarrow NP$ ;  
17. return  $P$ ;
```

Luồng hoạt động tương ứng với thuật toán tiến hoá:

- **Bước 1:** Khởi tạo quần thể ban đầu ngẫu nhiên P , rồi đánh giá fitness score của chúng
- **Bước 2:** Kiểm tra điều kiện dừng C
- **Bước 3 – Lai tạo:**
 - Giữ lại các phân tử tốt nhất (dựa trên fitness score) của quần thể hiện tại đưa vào thế hệ sau NP

- Thêm lần lượt các test case vào thể hệ mới đến khi đạt số cá thể tối đa bằng cách:
 - Chọn 2 cá thể cha p_1, p_2 trong thể hệ trước P theo hàm lựa chọn s_f
 - Tạo 2 cá thể con o_1, o_2 bằng cách crossover và mutation từ p_1, p_2
- **Bước 4 – Đánh giá:** Thuật toán sử dụng điểm fitness là tổng branch distance của tất cả các test case trong test suite
- **Bước 5 – Chọn:** Nếu cá thể có fitness score tốt nhất là 1 trong 2 cá thể con thì đưa cặp cá thể con vào thể hệ sau NP, ngược lại giữ lại cặp cá thể cha

3.2.5.2. Thuật toán MOSA

Thuật toán tiến hóa nói chung thường sẽ chỉ hướng đến việc tối ưu 1 mục tiêu (target) nhất định, nên với các bài toán mà yêu cầu đầu ra phải tối ưu nhiều mục tiêu cùng lúc thì sẽ gặp khó khăn. Do đó, thuật toán MOSA được tạo ra nhằm giải quyết các bài toán đa mục tiêu.

Dưới đây là mã giải cho thuật toán:

Input: Điều kiện dừng C , hàm fitness δ để đánh giá các cá thể, Kích thước quần thể p_s , hàm lai tạo c_p để tạo cá thể con, hàm đột biến cá thể m_p

Output: 1 tập hợp A chứa các cá thể tối ưu

```

1   $p \leftarrow 0$ ;
2   $N_p \leftarrow \text{GenerateRandomPopulation}(p_s)$ ;
3   $\text{PerformFitnessEvaluation}(\delta, N_p)$ ;
4   $A \leftarrow \{\}$ ;
5  while  $\neg C$  do
6       $N_o \leftarrow \text{GenerateOffspring}(c_f, c_p, m_p, N_p)$ ;
7       $R_t \leftarrow N_p \cup N_o$ ;
8       $r \leftarrow 0$ ;
9       $F_r \leftarrow \text{PreferenceSorting}(R_t)$ ;
10      $N_{p+1} \leftarrow \{\}$ ;
11     while  $|N_{p+1}| + |F_r| \leq p_s$  do
12          $\text{CalculateCrowdingDistance}(F_r)$ ;
13          $N_{p+1} \leftarrow N_{p+1} \cup F_r$ ;
14          $r \leftarrow r + 1$ ;
15      $\text{DistanceCrowdingSort}(F_r)$ ;

```

```

16    $N_{p+1} \leftarrow N_{p+1} \cup Fr$  with size  $ps - |N_{p+1}|$ ;
17   UpdateArchive(A,  $N_{p+1}$ );
18    $p \leftarrow p + 1$ ;
19   return A

```

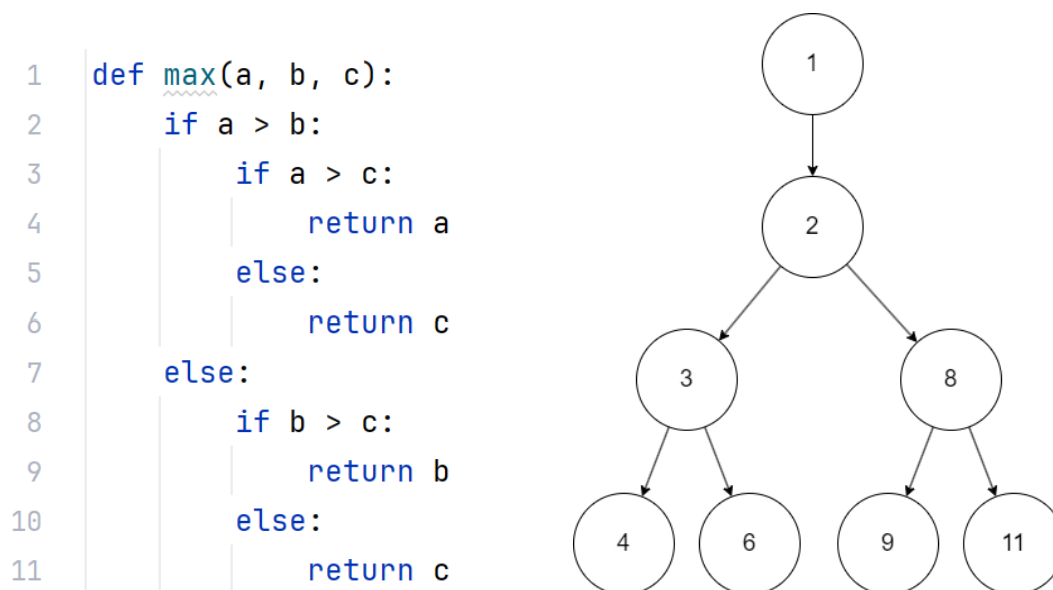
Luồng hoạt động tương ứng với thuật toán tiến hoá:

- **Bước 1:** Khởi tạo quần thể ban đầu ngẫu nhiên NP, rồi đánh giá fitness score của chúng. Tạo 1 tập archive A để chứa các test case phủ được từng nhánh (theo từng target) với số câu lệnh ít nhất
 - Thuật toán MOSA sẽ ưu tiên giữ lại các cá thể có fitness score tối ưu nhất cho tất cả các target (cực trị toàn cục), và loại bỏ các cá thể có fitness score cao trên 1 target nhưng thấp trên những target khác (cực trị cục bộ). Cần có tập archive A độc lập với quần thể NP để giữ lại các cá thể cực trị cục bộ này.
- **Bước 2:** Kiểm tra điều kiện dừng C
- **Bước 3 – Lai tạo:** Tạo các cá thể con No bằng cách crossover và mutation trên quần thể hiện tại N_p rồi gộp chung lại thành tập R_t
- **Bước 4 – Đánh giá:**
 - Tính fitness score của từng cá thể trong R_t :
 - Với mỗi 1 target, fitness score của cá thể trên target này tính bằng tổng của branch distance và approach level (số lượng nhánh chưa được phủ nằm giữa test case và target)
 - Từ đó ta được 1 tập giá trị fitness score trên toàn bộ target của cá thể đó
 - Xếp từng cá thể của R_t thành từng tập $F_0, F_1, \dots$ tương ứng với độ tối ưu đa mục tiêu của tập giá trị fitness score vừa tính ở trên.
- **Bước 5 – Chọn:** Lấy các cá thể vào quần thể mới theo thứ tự từ $F_0, F_1, \dots, F_n$ cho đến khi quần thể đạt số lượng tối đa.

3.2.5.3. Thuật toán DynaMOSA

Nhược điểm lớn nhất của MOSA là các target sẽ được coi như những đối tượng độc lập. Tuy nhiên các nhánh của chương trình trong thực tế thường có sự phụ thuộc lẫn nhau theo cấu trúc, chẳng hạn các lệnh điều kiện lồng nhau. DynaMOSA sử dụng 1 đồ thị phụ thuộc dòng điều khiển để quyết định việc lựa chọn target cho thuật toán tiến hoá.

Hai đỉnh trên đồ thị phụ thuộc dòng điều khiển có cạnh nối với nhau nếu đỉnh này phụ thuộc vào đỉnh kia.



Hình 6: Ví dụ về đồ thị phụ thuộc dòng điều khiển

DynaMOSA có cấu trúc tương tự MOSA, tuy nhiên ở bước lựa chọn cá thể, DynaMOSA sẽ chỉ lựa chọn target mà câu lệnh đó phụ thuộc trực tiếp vào nhánh đã được phủ từ trước trong bộ test. Với ví dụ trên hình, vì đỉnh 4 và đỉnh 6 phụ thuộc dòng điều khiển vào đỉnh 3, do đó đỉnh 4 và đỉnh 6 chỉ được coi là 1 target nếu bộ test đã phủ qua đỉnh 3.

Sau mỗi một vòng lặp, DynaMOSA sẽ cập nhật lại đồ thị phụ thuộc dòng điều khiển bằng cách duyệt lại các nhánh đã được thực thi.

3.2.5.4. Thuật toán MIO

Thuật toán nhiều mục tiêu độc lập MIO (Many Independent Objectives) (Arcuri 2017) được sinh ra nhằm khắc phục một số hạn chế của cách tiếp cận Whole Suite và MOSA với những bài toán có giới hạn về tài nguyên (bộ nhớ, thời gian).

MIO giải quyết vấn đề này bằng cách sử dụng quần thể động (dynamic population), cho phép linh hoạt thêm hoặc bớt các cá thể trong quá trình sinh ra đột biến, thay vì giữ nguyên một số lượng cố định.

Mã giả của thuật toán:

Input: Điều kiện dừng C , hàm fitness δ , kích thước quần thể N , số lần đột biến M , hàm mutation m_f , xác suất mutation m_p , xác suất chọn mẫu ngẫu nhiên R , thời điểm bắt đầu tìm kiếm tập trung F .

Output: Quần thể các cá thể tối ưu hoá A

```
1.  $Z \leftarrow \text{SetOfEmptyPopulations}()$ ;
2.  $A \leftarrow \{\}$ ;
3.  $p \leftarrow \text{null}$ ;
4.  $m \leftarrow 1$ ;
5. while  $\neg C$  do
6.   if  $p \neq \text{null} \wedge m < M$  then
7.      $p \leftarrow \text{Mutation}(m_f, m_p, p)$ ;
8.      $m \leftarrow m+1$ ;
9.   else if  $p = \text{null} \vee R > \text{RANDOM}(0,1)$  then
10.     $p \leftarrow \text{GenerateRandomIndividual}()$ ;
11.     $m \leftarrow 1$ ;
12.   else
13.     $p \leftarrow \text{SampleIndividual}()$ ;
14.     $p \leftarrow \text{Mutation}(m_f, m_p, p)$ ;
15.     $m \leftarrow 1$ ;
16.   forall  $t \in \text{ReachedTarget}(p)$  do
17.     if  $\text{isTargetCovered}(t)$  then
18.        $\text{updateArchive}(A, p)$ ;
19.        $Z \leftarrow Z \setminus \{Z_t\}$ ;
20.     else
21.        $Z_t \leftarrow Z \cup \{p\}$ ;
22.     if  $|Z_t| > N$  then
```

23. RemoveWorseTest(Z_t, δ);
24. UpdateParameters(F, R, N, M);
25. **return** A;

Luồng hoạt động tương ứng với thuật toán tiến hoá:

- **Bước 1:** Khởi tạo quần thể ban đầu ngẫu nhiên Z , rồi đánh giá fitness score của chúng. Tạo 1 tập archive A để chứa các test case phủ được từng nhánh (theo từng target) với số câu lệnh ít nhất (tương tự như thuật toán MOSA)
- **Bước 2:** Kiểm tra điều kiện dừng C
- **Bước 3 – Lai tạo:** Có 2 trường hợp xảy ra
 - Tạo 1 bộ test mới p ngẫu nhiên với xác suất R (hoặc khi archive rỗng)
 - Với xác suất $1-R$
 - Mutation bộ test có sẵn nếu số lần mutation chưa đạt tối đa
 - Ngược lại, lấy mẫu từ tập archive để tạo bộ test mới p
- **Bước 4 – Đánh giá + Bước 5 – Chọn:** Với mỗi mục tiêu t :
 - Nếu mục tiêu t đã đạt độ phủ như mong muốn:
 - Thêm p vào tập A .
 - Xóa các cá thể trong Z có liên quan đến t (nhánh đã đạt được độ phủ mong muốn).
 - Nếu mục tiêu t chưa đạt độ phủ:
 - Thêm p vào quần thể Z_t của target t .
 - Nếu Z_t có số lượng phần tử vượt quá kích thước tối đa N , xóa cá thể có fitness score kém nhất ra khỏi Z_t dựa trên hàm đánh giá δ . (Mặc định $\delta = 1 / (1+d)$ với d là branch distance)
- Các tham số như xác suất lấy mẫu R , số lượng đột biến tối đa M sẽ thay đổi dựa trên thời điểm bắt đầu tìm kiếm tập trung F . (Biến F biểu thị tỷ lệ phần trăm thời gian sử dụng hết ngân sách tìm kiếm, sau đó thuật toán sẽ chuyển từ giai đoạn tìm kiếm toàn diện sang giai đoạn tìm kiếm tập trung - focused search).

CHƯƠNG 4. ASSERTION & MUTATION

Assertion là một phần quan trọng trong khâu kiểm thử, sau khi sinh ra quần thể chứa các test case thì sẽ cần đến **assertion** để có thể kiểm tra tính đúng đắn của test case, và xem liệu các testcase đã hoàn thiện cho khâu export chưa. Trong đó, một **assertion** cơ bản sẽ so sánh 2 giá trị giữa actual output (giá trị kỳ vọng) và expected output (giá trị thật), liệu xem 1 module có trả về đúng giá trị kỳ vọng không.

Đối với Pynguin, mục tiêu đề ra không chỉ nhằm sinh ra **assertion**, mà sinh ra các regression assertion, kết hợp với mutants giúp lọc ra các test case phù hợp nhất để export.

Đoạn dưới đây sẽ tóm tắt các khái niệm chính mà nhóm tác giả đã đề ra trong bài báo, nhằm cụ thể hoá việc sinh regression assertion, cũng như giải thích cơ chế sinh ra chúng.

4.1. Định nghĩa và ví dụ

- **Regression Assertion:** Các đoạn check tự động giúp kiểm tra các tính năng đã có vẫn hoạt động đúng sau thay đổi, cập nhật hoặc sửa lỗi, thường gắn với 2 giá trị true hoặc false.
- **Assertable:** Các kiểu dữ liệu built-in trong Python, bao gồm các giá trị enum, các giá trị nguyên thủy built-in (int, str, bytes, bool, và None), và các giá trị kiểu collection chỉ chứa assertable (tuple, list, dict, và set). Với kiểu assertable, Pynguin có thể sinh ra các assertion trực tiếp bằng cách tạo các expected value để so sánh.
- **Non-assertable:** Các kiểu dữ liệu còn lại. Pynguin tạo assertion bằng cách sinh expected state và attribute. Đối với kiểu dữ liệu non-assertable collections, Pynguin dựng collection với size bằng size collection cần test, và so sánh. Nếu các cách trên không được, Pynguin sẽ tạo assertion với giá trị not None.
- **Mutant:** Các đoạn mã có thay đổi nhỏ về mặt syntax thông qua các mutation operators nhằm kiểm tra test case.

Ví dụ:

```
import queue

def divide(x: int, y: int) -> float:
    if y < 0:
        raise ValueError("y shall only be positive")
    return x / y

def queue():
    return queue.Queue()
```

Hình 7: Một đoạn mã để sinh test với assertion

```
import example as module0
import pytest

def test_case_0:
    int_0 = 1
    int_1 = 2
    float_0 = module0.divide(int_0, int_1)
    assert pytest.approx(0.5) == float_0

def test_case_1:
    var_0 = module_0.queue()
    assert var_0 is not None
```

Hình 8: Một số test case cho đoạn code trên

Giải thích:

- Với hàm divide:
 - Pynguin khởi tạo test case ban đầu với x, y ngẫu nhiên, sau đó chạy với hàm divide, và kết quả trả về là 0.5.
 - Pynguin sẽ chạy với cặp x, y này 2 lần nhằm đảm bảo assertion được tạo không phụ thuộc vào yếu tố môi trường hoặc bộ nhớ.
 - Vì x và y thuộc kiểu assertable, Pynguin có thể tạo assertion trực tiếp bằng cách tạo expected value để so sánh (0.5).
- Với hàm queue:
 - Pynguin nhận thấy kiểu dữ liệu trong hàm là 1 Queue, thuộc kiểu non-

assertable, nên Pynguin chỉ trả về assertion trong mọi trường hợp “is not None”

4.2. Exceptions

Pynguin cũng có thể tạo assertion cho các exception được raised. Tuy nhiên, Pynguin không phân biệt được khái niệm giữa checked và unchecked, giữa expected và unexpected exceptions, vì vậy Pynguin sẽ phân tích AST (Cây cú pháp trừu tượng) nhằm phát hiện các exception được raise nhưng không được catch, hoặc được nêu trong chú thích của hàm.

- Đối với expected exception, Pynguin sử dụng `pytest.raises` từ framework Pytest và thêm việc raise exception vào trong test case.
- Đối với unexpected exception, Pynguin sử dụng `pytest.mark.xfail` decorator cho những test case trả về unexpected exception, tức những test case trả về exception mà chương trình không đề cập đến.

Ví dụ:

```
import example as module0
import pytest

def test_case_0:
    int_0 = 1
    int_1 = -1
    with pytest.raises(ValueError):
        module0.divide(int_0, int_1)

@pytest.mark.xfail(strict=True)
def test_case_0:
    int_0 = 1
    int_1 = 0
    module0.divide(int_0, int_1)
```

Hình 9: Một số test case với exception cho đoạn code trên

Giải thích:

- Với hàm `divide`, trong đoạn code đã có đề cập việc raise exception đối với trường hợp `y` là số âm, vì vậy Pynguin sau khi phân tích đoạn code sẽ sử dụng `pytest.raises` đối với test case `x= 1` và `y = -1`, thể hiện rằng đoạn code này với test case này sẽ trả về 1 expected exception.

- Với test case dưới ($x = 1, y = 0$), vì trong đoạn code không đề cập đến việc raise exception với trường hợp chia cho số 0 nên Pynguin sẽ chạy test case bình thường, cho đến khi gặp lỗi. Pynguin sau đó sẽ gán thêm vào đoạn test case 1 decorator để báo cho người dùng rằng với test case này, khi thực thi sẽ sinh ra 1 unexpected exception, và để người dùng tự xử lý.

4.3. Cơ chế kiểm thử với assertion

1. Pynguin sẽ chạy mỗi test case 2 lần 1 cách random tránh assertions không ổn định, tức các assertion phụ thuộc vào các yếu tố ngoại lai, tạo ra các base assertion.
2. Pynguin sử dụng 1 biến thể của MutPy để sinh mutants bằng cách áp dụng các mutation operators. Sau đó, Pynguin thực thi test case với mutant, nếu kết quả thay đổi so với module ban đầu tức mutant bị “giết” và assertion đạt hiệu quả, nếu giữ nguyên tức mutant “sống” và assertion đó sẽ bị loại bỏ khỏi nhóm base assertion.
3. Chỉ các assertion giúp “giết” mutant được giữ lại, và qua nhiều lần thực hiện quá trình này, nhóm các assertion đạt hiệu quả cao nhất sẽ được sử dụng trong bộ test case.

CHƯƠNG 5. THỰC NGHIỆM

Cấu hình của thực nghiệm trong bài nghiên cứu như sau:

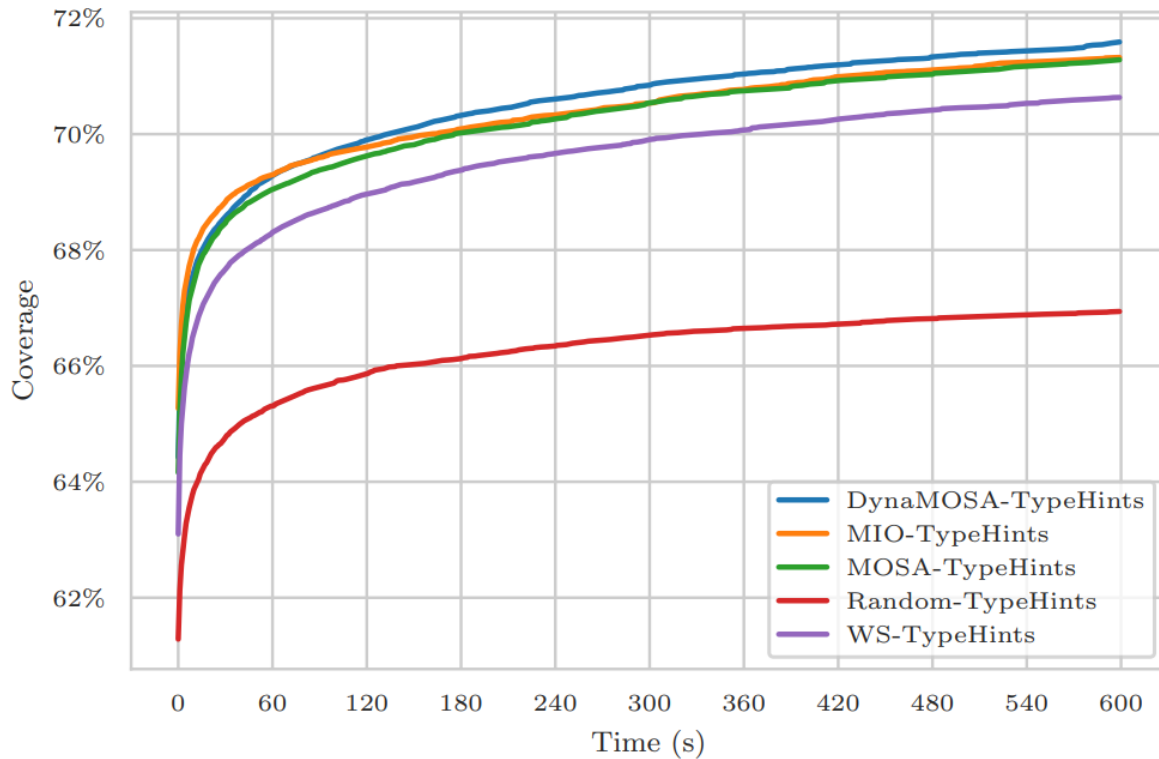
- Chạy 30 lần cho mỗi cấu hình thuật toán.
- Máy chủ cài đặt AMD EPYC 7443P CPU và 256GB RAM.
- Mỗi lần chạy sẽ chạy trên 1 lõi CPU và giới hạn 4GB RAM.
- Thời gian chạy 600s.

Từ kết quả chạy thực nghiệm, có 3 câu hỏi nghiên cứu được đưa ra:

- RQ1: Các phương pháp tạo kiểm thử với đoạn mã Python khác nhau thế nào?
- RQ2: Việc có sẵn thông tin kiểu dữ liệu ảnh hưởng thế nào đến việc tạo kiểm thử?
- RQ3: Các assertions được tạo ra có ảnh hưởng thế nào đến việc phát hiện lỗi?

5.1. RQ1: So sánh hiệu quả của các thuật toán được đề cập.

Dưới đây là biểu đồ thể hiện quan hệ giữa độ phủ và thời gian của các thuật toán.



Hình 10: Độ phủ mã theo thời gian với thông tin kiểu dữ liệu

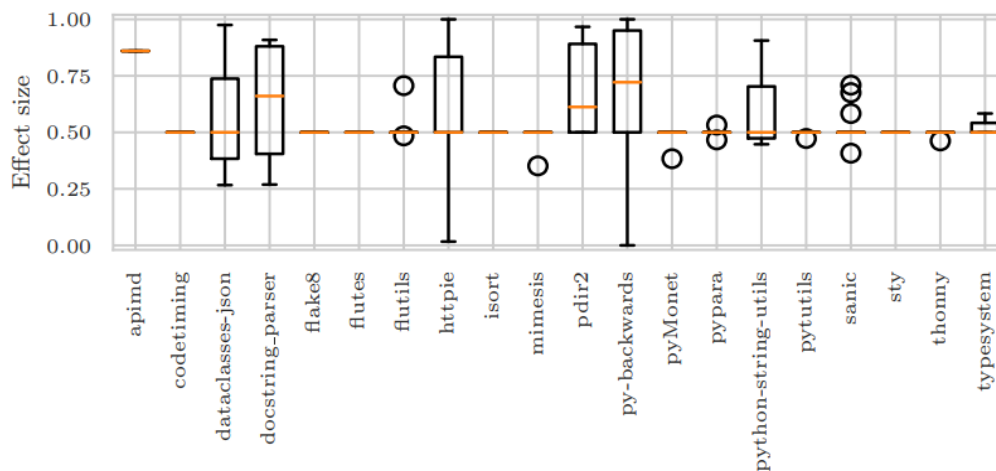
Trong biểu đồ trên, trong phút đầu tiên chạy thực nghiệm, thuật toán MIO đạt được phạm vi phủ mã cao nhất trước khi bị thuật toán DynaMOSA vượt qua. Sau đó, hiệu suất của thuật toán MIO bị giảm dần. Thuật toán MOSA hoạt động kém hiệu quả hơn một chút so với MIO. Với 2 thuật toán Whole-Suite và Random, hiệu suất hoạt động kém xa so với 3 thuật toán đầu, trong đó Random là thấp nhất.

Algorithm	With Type Hints		Without Type Hints	
	median (%)	mean (%)	median (%)	mean (%)
DynaMOSA	80.7	71.6 ± 30.5	76.5	69.4 ± 31.0
MIO	80.0	71.3 ± 30.7	75.0	68.4 ± 31.3
MOSA	80.0	71.3 ± 30.8	76.7	68.7 ± 31.7
Random	71.4	66.9 ± 31.5	66.7	62.6 ± 32.9
WS	80.0	70.6 ± 30.7	71.4	67.5 ± 31.5

Bảng 1. Giá trị phủ trung vị và giá trị phủ trung bình với độ lệch chuẩn giữa các thuật toán khi có và không có thông tin kiểu dữ liệu

Dựa vào bảng trên, khi có thông tin kiểu dữ liệu, độ phủ mã tăng lên một chút. Các thuật toán tạo test trong Python mang lại giá trị phạm vi phủ mã từ 66,9% đến 71,6% trung bình. Trong đó, thuật toán DynaMOSA hoạt động tốt nhất, sau đó đến MIO, MOSA, Whole-Suite, thuật toán Random đạt độ phủ thấp nhất.

5.2. RQ2: Ảnh hưởng của thông tin kiểu dữ liệu



Hình 11: Phân phối effect-size với mỗi project (DynaMOSA)

Ở thực nghiệm này, xuất hiện khái niệm về thống kê Vargha-Delaney A_{12} [1]. Thống kê này đo lường khả năng một giá trị ngẫu nhiên từ một nhóm này sẽ cao hơn giá trị ngẫu nhiên từ nhóm khác, dựa trên sự sắp xếp các giá trị.

Nếu $A_{12} = 0.500$, hai cấu hình có mức độ bao phủ tương đương nhau (không có sự khác biệt đáng kể).

Nếu $A_{12} > 0.500$, cấu hình C1 có mức độ bao phủ cao hơn cấu hình C2.

Nếu $A_{12} < 0.500$, cấu hình C2 có mức độ bao phủ cao hơn cấu hình C1.

Biểu đồ box plot trên thể hiện độ đo A_{12} khi áp dụng thuật toán DynaMosa có kiểu dữ liệu và DynaMosa không có kiểu dữ liệu cho các project khác nhau. Ta có thể thấy giá trị trung vị của các project đều lớn hơn 0.5 do vậy có thể kết luận rằng thông tin kiểu dữ liệu giúp đạt được độ bao phủ cao hơn.

5.3. RQ3: Hiệu quả Assertion trong việc phát hiện lỗi

Nhóm tác giả sử dụng mutation score là đơn vị chính nhằm đánh giá độ hiệu quả của assertion. Mutation score được tính bằng tỉ lệ giữa số lượng mutant bị “giết” trên tổng số mutant sinh.

Để trả lời câu hỏi này, 105/134 python module từ việc kiểm nghiệm RQ1 và RQ2 được tái sử dụng, sử dụng thuật toán DynaMOSA - TypeHints, được đánh giá là thuật toán đạt hiệu quả tốt nhất. Trong khi đó, 30 module còn lại sau 30 lần chạy trả về kết quả lỗi nên sẽ không được sử dụng (Lý do 30 module lỗi có thể do sự khác biệt phiên bản Python của MutPy và Pynguin).

5.3.1. Một số kiến thức liên quan

5.3.1.1. Hệ số tương quan Pearson

Khi chúng ta cần quan sát và đưa ra nhận xét về mối tương quan giữa hai giá trị, chúng ta có thể sử dụng một số loại hệ số nhằm đưa ra cái nhìn tổng quan về xu hướng giữa chúng, một trong số đó là hệ số tương quan Pearson.

Theo Gayen (1951), đối với trường hợp hai biến giá trị định lượng là **tuyến tính** với nhau (tức mỗi quan hệ giữa chúng có thể được biểu diễn theo xu hướng đường thẳng), có thể sử dụng hệ số tương quan Pearson.

Hệ số này cho phép chúng ta xác định độ mạnh và hướng của mối quan hệ giữa hai giá trị tuyến tính, thông qua công thức:

$$r = (n\sum xy - (\sum x)(\sum y)) / \sqrt{([n\sum x^2 - (\sum x)^2][n\sum y^2 - (\sum y)^2])}$$

Trong đó:

- x: các điểm giá trị của biến đầu;
- y: các điểm giá trị của biến sau;
- n: tổng số các cặp điểm giá trị trong mẫu

Tùy vào giá trị trả về của r ta có thể đưa ra đánh giá về mức tương quan giữa 2 giá trị:

Hệ số r	Strength	Chiều
$r \geq 0.5$	Mạnh	Dương
$0.3 \leq r < 0.5$	Tương đối	Dương
$0 < r < 0.3$	Yếu	Dương
0	Không	Không
$-0.3 < r < 0$	Yếu	Âm
$-0.5 \leq r < -0.3$	Tương đối	Âm
$r \leq -0.5$	Mạnh	Âm

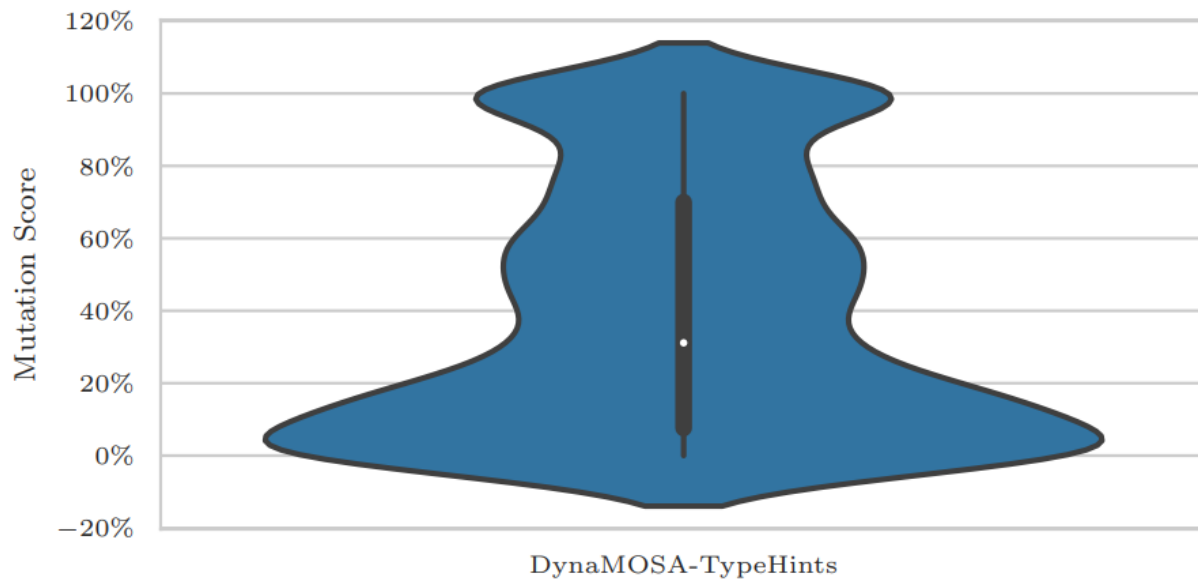
5.3.1.2. Giá trị p-value

p-value (probability value), thể hiện xác suất giả thuyết có thể xảy ra trong thực tế, và có thể được quan sát. p-value càng nhỏ tức trường hợp giả thuyết có thể xảy ra càng thấp, và giả thuyết thay thế sẽ có xác suất lớn xảy ra.

Trong đó, ta sẽ đặt ra 2 giả thuyết, một giả thuyết gốc (H_0) và một giả thuyết thay thế (H_1). Ta có thể tính p-value thông qua một số phương pháp như phân phối chuẩn z, phân phối t, phân phối chi - bình phương, etc. Và từ kết quả trả về ta có thể kết luận được liệu giả thuyết gốc có thể được loại bỏ và sử dụng giả thuyết thay thế.

Ngưỡng thông thường được chấp nhận cho việc loại bỏ giả thuyết gốc là 0.05 (5%).

5.3.2. Kết quả thực nghiệm

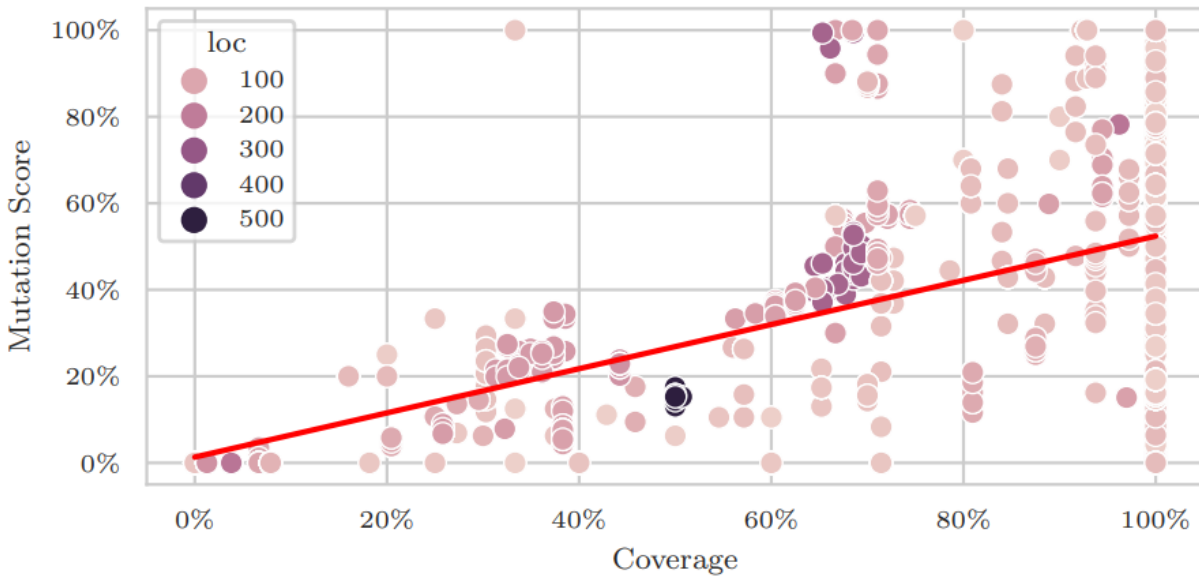


Hình 12: Phân bố mutation score đối với configurations trên

Dựa vào đồ thị, ta có thể thấy rằng:

- Kết quả mutation score chạy từ 0 - 100%, tuy nhiên mật độ từ 0 - 30% và 80 - 100% dày hơn so với các khoảng khác, cho thấy Pynguin có khả năng diệt mutant với độ hiệu quả tối đa.
- Chấm tròn trắng ở giữa thể hiện trung vị của phân bố chỉ rơi vào khoảng 30%, nói lên rằng vẫn có nhiều chỗ để cải thiện nhằm tăng độ hiệu quả.

Từ đó, có thể đưa ra giả thuyết về mối liên hệ tỉ lệ thuận giữa mutation score và độ phủ, rằng chỉ có những đoạn statement được mutated trong chương trình được phủ tới mới có thể được phát hiện.



Hình 13: Mối tương quan giữa mutation score và độ phủ

Trong đó, hệ số tương quan Pearson được đánh giá với mức $r = 0.45$, 1 hệ số dương và nằm giữa 0.3 và 0.5, cho thấy một mối tương quan tương đối chặt giữa mutation score và độ phủ. Ngoài ra, p-value được đánh giá với mức 5.53×10^{-157} , và nhỏ hơn so với ngưỡng loại bỏ giả thuyết 0.05. Điều đó cho thấy rằng có một mối liên hệ chặt chẽ giữa 2 giá trị trên.

Tuy nhiên, có một số trường hợp cho thấy mặc dù 100% mutation score nhưng chỉ được 30% coverage. Điều này có thể được giải thích bằng việc Pynguin chỉ có thể sinh được mutant và kiểm thử trên những mutant đó đến đoạn code được phủ. Đối với 1 đoạn code được kiểm thử, chỉ đoạn code được phủ mới có thể sinh mutant, vì vậy 100% đoạn code được phủ thì có thể đạt tỉ lệ score 100%, nhưng chỉ là 100% của 30% tổng đoạn code, 70% còn lại chưa được khám phá, dẫn tới một số trường hợp cá biệt.

Tổng kết:

Các thí nghiệm chỉ ra Pynguin có khả năng sinh assertion với độ hiệu quả cao, với mutation score có thể đạt tới 100%. Tuy nhiên vẫn cần cải thiện nhằm tăng tính nhất quán và độ ổn định cho đầu ra.

CHƯƠNG 6. KẾT LUẬN

6.1. Hạn chế Pynguin

Pynguin, vì đang trong quá trình phát triển và chỉ đang là một prototype, sẽ gặp một số vấn đề và hạn chế trong việc kiểm thử, trong đó:

1. Cơ chế cô lập:

- a. Một số module sử dụng các hàm sys, ví dụ như hàm sys.exit(). Điều này khiến cho Pynguin khi thực thi đến câu lệnh sẽ dừng trình dịch, đồng thời dừng việc thực thi của Pynguin. Trong tương lai, vấn đề này có thể được khắc phục bằng cách tạo một môi trường file ảo hoặc các hàm mock call tương tự như sys.exit.

2. Cơ chế động của Python:

- a. Vì bản chất ngôn ngữ Python là ngôn ngữ động, việc xử lý các đối tượng từ cùng 1 lớp vẫn có thể gây khó khăn cho Pynguin, nhất là trong quá trình thực thi khi 1 lớp có thể có thêm các method, như hàm setattr() hay __getattr__, điều này khiến phân tích kiểu tĩnh gặp khó khăn.

3. Chưa hỗ trợ các construct:

- a. Một số construct như Generator, Decorator, HOC (Higher Order Function) chưa được hỗ trợ bởi Pynguin vì kiểu dữ liệu khó khăn để xác định. Điều này phụ thuộc vào tính chất của các construct, ví dụ như các hàm Generator sẽ phải làm việc với kiểu yield, hay HOC sẽ làm việc với kiểu dữ liệu là các hàm callback, điều mà Pynguin không thể hỗ trợ được.

4. Khả năng phân biệt kiểu:

- a. Khả năng phân biệt kiểu của Pynguin cũng cần được nghiên cứu thêm, vì bản chất của Python là ngôn ngữ động nên việc khai báo các kiểu dữ liệu như any, unknown, undefined sẽ không gặp error lúc runtime. Sau khi thực thi các kiểu đó mới được xác định cụ thể. Tuy nhiên Pynguin chưa thể phân biệt tại thời điểm runtime các kiểu dữ liệu tổng quát sẽ được chuyển đổi về kiểu nào. Ví dụ, một kiểu List[Tuple[Any,...]] và kiểu List[Tuple[int, str]],

Pynguin không thể đọc được kiểu nào sẽ là kiểu nào lúc thực thi, và hiện tại cũng chưa có các hàm built-in nhằm hỗ trợ việc này.

6.2. Nhận xét

Trong thời đại ngày nay, việc kiểm thử đóng một vai trò vô cùng quan trọng trong việc phát triển và giám sát sản phẩm. Trong đó, các phần mềm sinh ca kiểm thử tự động, giúp tăng hiệu suất phát hiện lỗi, càng ngày càng được phát triển và tối ưu hoá, tích hợp vào các ngôn ngữ khác nhau. Tuy nhiên, với các ngôn ngữ kiểu động như Python, việc có một thư viện kiểm thử có thể đạt hiệu quả cao vẫn đang trong quá trình phát triển, và mặc dù Pynguin đã có những bước đầu tiếp cận đến vấn đề này, các định hướng phát triển trong tương lai nhằm giải quyết các hạn chế còn tồn đọng sẽ vẫn cần thêm thời gian để xem xét. Mặc dù vậy, với những gì Pynguin đã đạt được qua việc thực nghiệm, tiềm năng của Pynguin trong tương lai sẽ có thể giúp việc kiểm thử Python được dễ dàng hơn, và từ đó mở ra các hướng nghiên cứu cho các ngôn ngữ động khác.

CHƯƠNG 7. PHỤ LỤC

7.1. Ý nghĩa của kiểm thử

- Giúp phát hiện ra lỗi sớm: Nếu phát hiện ra lỗi muộn, phần mềm có khả năng phải thiết kế lại toàn bộ những module liên quan đến lỗi đó, làm mất nhiều thời gian và chi phí hơn cho quá trình phát triển.
- Cải thiện chất lượng và độ tin cậy của sản phẩm: Kiểm thử phần mềm giúp tăng sự tin tưởng và hài lòng của khách hàng bằng cách đảm bảo ứng dụng không có lỗi. Bằng các phương pháp kiểm thử hiệu năng, kiểm thử bảo mật và các phương pháp khác, kiểm thử phần mềm giúp tăng độ tin cậy của phần mềm.
- Giúp thuận tiện cho việc mở rộng: Việc kiểm thử phi chức năng giúp đội phát triển xác định các vấn đề về khả năng mở rộng cũng như các rủi ro có thể làm cho phần mềm ngừng hoạt động khi tiến hành mở rộng.
- Tiết kiệm thời gian và chi phí: Sau khi phần mềm được triển khai, việc truy vết và giải quyết các vấn đề sẽ rất khó khăn, dẫn đến tốn kém hơn cả về thời gian và chi phí. Do đó, việc kiểm thử phần mềm hiệu quả ngay trong quá trình phát triển là vô cùng cần thiết.

7.2. Cách sinh giá trị Assertion

Trừ khi có tài liệu về các hành vi và kết quả mong đợi trả về của các code module, sẽ không thể biết chính xác giá trị kỳ vọng trả về của module. Thay vào đó, Pynguin sẽ chạy các test case ban đầu 2 lần, và nếu giá trị trả về trong 2 lần giống nhau thì Pynguin sẽ note lại giá trị đó là giá trị “kỳ vọng” của module đó. Và sau khi sinh ra các mutant, Pynguin sẽ so sánh giá trị sau khi được mutated với giá trị “kỳ vọng” nhằm kiểm tra độ hiệu quả của Assertion trong việc “giết” các mutants.

7.3. Thống kê về các kiểu dữ liệu hỗ trợ trong Pynguin

Nhóm sử dụng đoạn mã sau để kiểm tra mức độ hỗ trợ của Pynguin với mỗi kiểu dữ liệu, trong đó type có thể là 13 kiểu dữ liệu sau:

- Kiểu dữ liệu nguyên thủy: int, float, complex, bool, str, bytes
- Kiểu None
- Collection: list, tuple, dict, set

- Kiểu enum và kiểu Class (tham số truyền vào là 1 đối tượng của Class khác)

Thực nghiệm được thực hiện trên cả 5 thuật toán (Random, Whole suite, MOSA, DynaMOSA, MIO), có và không sử dụng Type Hint. Điều kiện dừng của thuật toán được đặt mặc định: tối đa 600s.

```
class TypeTest:
    def __init__(self, a):
        self.a = a

    def check_type(self, x):
        if not isinstance(self.a, type):
            self.a = x
```

Hình 14: Đoạn mã kiểm tra kiểu dữ liệu hỗ trợ của Pynguin.

Sau khi chạy Pynguin 3 lần với từng cặp thuật toán và kiểu dữ liệu, nhóm đưa ra kết luận: Pynguin hỗ trợ 13 kiểu dữ liệu trên như bài báo đã nêu, tuy nhiên không hỗ trợ các kiểu dữ liệu khác. Do tham số được sinh là ngẫu nhiên nên Pynguin có xác suất cao không thể sinh được test case phủ nhánh nếu biểu thức điều kiện của nhánh đó không phải biểu thức kiểm tra kiểu như trên.

TÀI LIỆU THAM KHẢO

- [1] Neumann, G., Harman, M., Poulding, S. (2015). Transformed Vargha-Delaney Effect Size. In: Barros, M., Labiche, Y. (eds) Search-Based Software Engineering. SSBSE 2015. Lecture Notes in Computer Science(), vol 9275. Springer, Cham.
- [2] Kumar, G. A. (1951, June 1). The Frequency Distribution of the Product-Moment Correlation Coefficient in Random Samples of Any Size Drawn from Non-Normal Universes. *Biometrika*.
- [3] Turney, S. (2022, May 13). *Pearson Correlation Coefficient (r) | Guide & Examples*. Scribbr. Retrieved December 18, 2024, from <https://www.scribbr.com/statistics/pearson-correlation-coefficient/>
- [4] Bevans, R. (2020, July 16). *Understanding P-values | Definition and Examples*. Scribbr. Retrieved December 18, 2024, from <https://www.scribbr.com/statistics/p-value/>
- [5] Fraser, G., & Arcuri, A. (2012, February 16). Whole Test Suite Generation. *IEEE Transactions on Software Engineering*, 39(2), 276–291. <https://ieeexplore.ieee.org/document/6152257>
- [6] Annibale Panichella , Fitsum Meshesha Kifetew , Paolo Tonella
Automated Test Case Generation as a Many-Objective Optimisation Problem with Dynamic Selection of the Targets
- [7] Arcuri, Andrea. (2017). Many Independent Objective (MIO) Algorithm for Test Suite Generation. 3-17. 10.1007/978-3-319-66299-2_1.
- [8] Annibale Panichella, Fitsum Meshesha Kifetew, Paolo Tonella
Reformulating Branch Coverage as a Many-Objective Optimization Problem